

Assignment 3: Instructions for Running the Pipeline

1. Prerequisites

Local Setup

- **Python 3.11+** (venv recommended)
- **MiniStack** installed: ``pip install ministack``
- **AWS CLI** available locally
- **Git** for version control

Cluster Setup (LBD)

- MiniStack pre-installed on cluster
- SSH access to cluster
- Dataset available at ``hdfs:///dic_shared/amazon-reviews/full/reviews_devset.json``

2. Local Development Workflow

Step 1: Install Dependencies

```
cd /path/to/Assignment3
python3 -m venv venv
source venv/bin/activate # or venv\Scripts\activate on Windows
pip install -r src/requirements.txt
```

Step 2: Start MiniStack (Terminal 1)

```
ministack
```

This runs the fake AWS services at ``http://localhost:4566``. Keep this terminal open throughout development.

Step 3: Provision All Resources (Terminal 2)

```
bash src/run.sh
```

This script (idempotent, safe to re-run) creates:

- 5 S3 buckets (``reviews-ingest``, ``reviews-preprocessed``, ``reviews-profanity``, ``reviews-scored``, ``results-export``)
- 2 DynamoDB tables (``Reviews`` with PK ``reviewId``, ``Customers`` with PK ``reviewerID``)
- 12 SSM parameters (bucket names, table names, thresholds)
- 5 Lambda functions (preprocess, profanity, sentiment, aggregate, report)
- 4 S3→Lambda event notifications (the pipeline chain)

****Output:**** "Provision complete" message. All resources are now ready.

Step 4: Run a Smoke Test (1 Review)

```
python src/loader.py data/reviews_devset.json 1
```

Wait 2–3 seconds for the Lambda chain to execute, then verify:

```
aws --endpoint-url=http://localhost:4566 dynamodb scan --table-name Reviews --select COUNT
```

Should show `Count: 1`.

Step 5: Process a Batch (100 Reviews)

```
python src/loader.py data/reviews_devset.json 100 --batch-size 50 --batch-delay 20
```

This uploads reviews in batches with a 20-second pause between batches, giving Lambda time to drain.

Step 6: Process the Full Devset (78,829 Reviews)

```
python src/loader.py data/reviews_devset.json --batch-size 200 --batch-delay 30
```

This takes ~2–4 hours locally. Monitor progress:

```
# Check how many reviews have landed in the Reviews table
while true; do
aws --endpoint-url=http://localhost:4566 dynamodb scan --table-name Reviews --select COUNT
sleep 10
done
```

Step 7: Generate Results

```
python src/generate_results.py
```

This:

- Scans the `Reviews` table (devset only)
- Counts sentiment distribution
- Counts profanity failures
- Scans `Customers` table for banned users
- Writes `results.json`
- Prints a summary to stdout

****Expected output:****

```
{
"sentiment": {
"positive": 67908,
"neutral": 1282,
```

```
"negative": 9639
},
"profanityFailed": 6983,
"bannedUsers": ["A13QTZ8CIMHHG4", "A2EDZH51XHFA9B", ...]
}
```

Step 8: Run Integration Tests

```
cd src
pytest tests/test_integration.py -v # smoke tests
pytest tests/test_integration_edge.py -v # edge cases
RUN_STRESS=1 pytest tests/test_stress.py -v -s # optional: full devset
```

3. Cluster Workflow (LBD)

One-Command End-to-End Run

```
# Terminal 1: start MiniStack
ministack
```

```
# Terminal 2: run everything
bash src/run_cluster.sh
```

This **automatically**:

1. Checks MiniStack is reachable at `http://localhost:4566`
2. Fetches `reviews\_devset.json` from HDFS (if not already local)
3. Provisions all resources via `run.sh`
4. Loads all 78,829 reviews in batches of 200 with 30s pauses
5. Waits for all reviews to reach the `Reviews` table (polling every 10s)
6. Generates results and writes `results.json`
7. Prints a final summary

Time estimate: ~2–4 hours to process 78,829 reviews

Custom Overrides

```
# Smaller batches (if memory errors occur)
BATCH_SIZE=100 BATCH_DELAY=60 bash src/run_cluster.sh

# Smoke test with fewer reviews
REVIEW_COUNT=500 BATCH_SIZE=100 BATCH_DELAY=10 bash src/run_cluster.sh

# Custom MiniStack endpoint (if not localhost:4566)
MINISTACK_ENDPOINT=http://custom:4566 bash src/run_cluster.sh
```

Resumable Runs

If processing is interrupted:

- A checkpoint file saves state every `CHECKPOINT\_INTERVAL` seconds (default 30)
- Checkpoint files are stored in `checkpoints/segment\_NNN.json`
- Re-running `run\_cluster.sh` automatically skips reviews already processed and continues from where it left off
- The final `results.json` is computed by **summing** all completed segments (durable on disk)

This design survives a full MiniStack restart or SSH session death — just re-run the same command.

4. Troubleshooting

"ERROR: MiniStack not reachable"

- Ensure `ministack` is running in Terminal 1
- Check: `curl -s http://localhost:4566/\_localstack/health`

"ERROR: Lambda timeout / DynamoDB ConditionalCheckFailedException"

- This is normal if the Lambda chain is slower than the loader
- Solution: increase `--batch-delay` (e.g., 60 seconds)
- Example: `python src/loader.py data/reviews\_devset.json --batch-size 100 --batch-delay 60`

"Reviews not appearing in DynamoDB"

- Check if reviews are stuck in intermediate buckets (preprocessed, profanity, scored):

```
aws --endpoint-url=http://localhost:4566 s3 ls s3://reviews-scored/
```

- If reviews are in scored but not in Reviews table: L4 (aggregate) may have crashed
- Check CloudWatch logs (MiniStack logs locally): `docker logs`

"ReviewId already exists" (ConditionalCheckFailedException on 2nd run)

- This is **expected** — the idempotency gate is working
- Use `--skip` to skip already-processed reviews: `python src/loader.py data/reviews\_devset.json --skip 500`

Full MiniStack Restart Needed

- Kill MiniStack in Terminal 1 (Ctrl+C)
- Wait 5 seconds
- Restart `ministack` and re-run `bash src/run.sh`
- This recreates all buckets and tables (clean slate)

5. Key Files Reference

File	Purpose
------	---------

-----	-----
`src/run.sh`	Idempotent provisioning script (SSM, S3, DynamoDB, Lambda, notifications)
`src/run_cluster.sh`	Cluster end-to-end runner (fetch data, provision, load, checkpoint, finalize)
`src/loader.py`	Batch loader: reads reviews_devset.json, uploads to S3
`src/generate_results.py`	Query DynamoDB → results.json
`src/lambda/preprocess/handler.py`	L1: tokenize + lemmatize
`src/lambda/profanity/handler.py`	L2: profanity check
`src/lambda/sentiment/handler.py`	L3: sentiment analysis (VADER + star rating)
`src/lambda/aggregate/handler.py`	L4: DynamoDB write + ban logic (idempotent)
`src/lambda/report/handler.py`	L5: on-demand results query
`src/common/config.py`	SSM parameter getter (used by all Lambdas)
`src/common/s3_events.py`	S3 event parser + envelope read/write (used by all Lambdas)
`src/tests/test_integration.py`	Happy-path + ban tests
`src/tests/test_integration_edge.py`	Edge case tests (null overall, empty text, idempotency)
`src/tests/test_stress.py`	Full devset stress test (optional, `RUN_STRESS=1`)
`CONTRACT.md`	Frozen interface: SSM scheme, envelope shape, idempotency rules
`results.json`	Final results (sentiment counts, profanity failures, banned users)

6. Lambda Signatures

All Lambdas follow the same AWS signature:

```
def handler(event, context):
    # event = S3/DynamoDB event (dict with bucket, key, etc.)
    # context = AWS Lambda context (request ID, time left, etc.)
    # return = {"statusCode": 200} or error
    ...
    return {"statusCode": 200}
```

The pipeline ****self-chains****: each Lambda's last action is writing to S3, which triggers the next Lambda.

7. Configuration (SSM Parameters)

All operational values are stored in SSM Parameter Store (namespace `/dic-a3/...`). To view:

```
aws --endpoint-url=http://localhost:4566 ssm get-parameters-by-path --path /dic-a3 --recursive
```

To change a value (e.g., ban threshold):

```
aws --endpoint-url=http://localhost:4566 ssm put-parameter \
--name /dic-a3/config/ban-threshold \
--value 5 \
--overwrite
```

Re-run Lambdas — they read SSM at cold start.

8. Monitoring & Debugging

See Lambda Logs (MiniStack Local)

```
# Find the MiniStack container
docker ps | grep localstack
```

```
# View logs
docker logs -f
```

Query S3 Buckets

```
aws --endpoint-url=http://localhost:4566 s3 ls s3://reviews-scored/
aws --endpoint-url=http://localhost:4566 s3 cp s3://reviews-scored/reviews/.json - | jq .
```

Query DynamoDB

```
# Count items in Reviews table
aws --endpoint-url=http://localhost:4566 dynamodb scan --table-name Reviews --select COUNT

# Get one review
aws --endpoint-url=http://localhost:4566 dynamodb get-item \
--table-name Reviews \
--key '{"reviewId":{"S":"abc123..."}}'

# List banned users
aws --endpoint-url=http://localhost:4566 dynamodb scan \
--table-name Customers \
--filter-expression 'banned = :t' \
--expression-attribute-values '{":t":{"BOOL":true}}'
```

9. Performance Notes

- **Local:** ~1–2 reviews per second (depends on machine)
- **Full devset (78,829):** ~2–4 hours on a typical laptop
- **Cluster:** ~2–3 times faster (more CPUs, better network)

Memory usage per Lambda invocation: ~50–100 MB (NLTK + profanity filter loaded at cold start).

10. Cleanup

Stop MiniStack

In Terminal 1: Ctrl+C

Delete Local Checkpoint Files (Cluster)

```
rm -rf checkpoints/
```

Reset Everything (next `run.sh` will recreate)

No manual cleanup needed. `run.sh` and `run\_cluster.sh` are idempotent: re-running deletes and recreates all resources.